

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет инженерно-экономический
Кафедра экономической информатики
Дисциплина «Программирование сетевых приложений»

«К ЗАЩИТЕ ДОПУСТИТЬ»

Руководитель курсового проекта
ассистент кафедры экономической
информатики

_____. А.С. Купрейчик
_____.2023

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему:

**«ПРОГРАММА ВЕДЕНИЯ СКЛАДСКОГО УЧЁТА И РЕГИСТРАЦИИ
ЗАКАЗОВ»**

Выполнил студент группы 124403
А. С. Мильто

(подпись студента)

Курсовой проект представлен на
проверку _____.2023

(подпись студента)

Минск 2023

СОДЕРЖАНИЕ

Введение	5
1 Организация деятельности складского учёта и регистрации заказов	7
1.1 Обзор предметной области	7
1.2 Обзор программных аналогов	8
2 Постановка задачи на разработку программного средства по ведению складского учёта и регистрации заказов и обзор методов её решения	11
3 Описание процесса ведения учёта товара на складе	13
4 Информационная модель системы и ее описание	18
5 Модели представления системы учёта товаров на складе на основе UML	20
5.1 Спецификация вариантов использования системы	20
5.2 Описание диаграммы последовательности системы	21
5.3 Описание диаграммы состояний	22
5.4 Описание диаграммы компонентов	23
5.5 Описание диаграммы развертывания	24
5.6 Описание диаграммы классов	24
6 Обоснование выбора программных средств разработки	26
7 Описание алгоритмов, реализующих бизнес-логику системы	30
7.1 Схема алгоритма клиент-серверного взаимодействия	30
7.2 Схема алгоритма работы администратора	31
7.3 Схема алгоритма добавления информации о товаре	32
8 Руководство пользователя	33
9 Результат тестирования системы учета товаров и регистрации заказов	45
Заключение	48
Список использованных источников	49
Приложение А (обязательное) Отчёт о проверке на заимствования в системе «Антиплагиат»	50
Приложение Б (обязательное) Листинг кода алгоритмов, реализующих бизнес-логику ..	51

ВВЕДЕНИЕ

В настоящее время компьютер является незаменимой частью любого обучающего либо производственного процесса. В связи с всё нарастающим потоком информации и надобностью структурировать его появилась потребность в создании программ, способных обрабатывать большие объёмы данных.

Компьютеры стали универсальными средствами хранения, получения и обработки информации любого вида. Они стали неизменными помощниками во всех сферах человеческой жизни.

Тема курсового проекта – «Программа ведения складского учёта и регистрации заказов».

Учет товаров на складе включает в себя следующие функции:

- хранение информации о товарах, включая название, артикул, описание, цену, количество и другие характеристики;
- отслеживание движения товаров на складе, включая поступление товаров, перемещение внутри склада и отгрузку клиентам;
- определение актуального количества товаров на складе и их местонахождения;
- мониторинг срока годности или других важных характеристик товаров;
- генерация отчетов о наличии товаров, движении и других аспектах учета.

Регистрация заказов от клиентов также включает несколько этапов:

- ввод данных о заказе в систему учета, включая информацию о клиенте, выбранных товарах, количестве и других деталях заказа;
- создание соответствующей записи о заказе в системе учета;
- передача информации о заказе на склад для сборки и отгрузки;
- отслеживание статуса заказа и его выполнение в соответствии с требованиями клиента;
- обновление информации о заказе в системе учета по мере его выполнения.

Программа ведения складского учета и регистрации заказов должна обладать следующими возможностями:

- создание и обновление базы данных товаров, включая информацию о каждом товаре, его характеристиках и количестве на складе;
- возможность быстрого и точного отслеживания движения товаров на складе, включая поступление, перемещение и отгрузку;

– ввод и обработка данных о заказах от клиентов, включая информацию о клиентах, выбранных товарах и деталях заказа;

– генерация отчетов о наличии товаров на складе, движении товаров и выполнении заказов.

Целью курсового проекта является реализация программного решения, которое позволит эффективно вести учёт товаров на складе, автоматизировать процесс регистрации и обработки заказов, а также оптимизировать работу с поставщиками и клиентами.

Поставленная цель потребовала решения следующих задач:

– изучить организацию деятельности склада;

– определить задачи на разработку программного средства складского учёта товаров и регистрации заказов;

– описать процесс складского учёта;

– разработать информационную модель учёта программного средства складского учёта и регистрации заказов и системы её описания;

– разработать модели представления складского учёта и регистрации заказов на основе UML;

– обосновать выбор программных средств разработки;

– описать алгоритмы, реализующие бизнес-логику системы;

– написать руководство пользователя;

– протестировать программный продукт.

В результате использования программы ведения складского учета и регистрации заказов компания сможет повысить эффективность своей работы, обеспечить точность и актуальность информации о товарах, контролировать движение товаров на складе и обрабатывать заказы клиентов быстро и эффективно. Это поможет компании удовлетворить потребности клиентов, минимизировать время доставки и снизить вероятность ошибок. Кроме того, программа позволит компании более точно планировать свою деятельность, определять популярные товары и спрос на них, что поможет принимать обоснованные решения о закупках и управлении запасами. В итоге, использование программы ведения складского учета и регистрации заказов способствует повышению конкурентоспособности компании и удовлетворению потребностей клиентов.

1 ОРГАНИЗАЦИЯ ДЕЯТЕЛЬНОСТИ СКЛАДСКОГО УЧЁТА И РЕГИСТРАЦИИ ЗАКАЗОВ

1.1 Обзор предметной области

Учёт товаров на складе и регистрация заказов являются важной составляющей эффективной работы любой компании, занимающейся продажей товаров. Он включает в себя процессы отслеживания, контроля и учёта товаров, а также регистрацию и обработку заказов от клиентов.

Одной из основных задач учёта товаров на складе является поддержание актуальной информации о наличии товаров. Для этого используются различные методы и инструменты, такие как штрих-коды, RFID-метки или системы автоматической идентификации товаров. С помощью этих инструментов можно быстро и точно определить количество и местонахождение каждого товара на складе.

Другим важным аспектом учета товаров на складе является контроль за движением товаров. Это включает в себя отслеживание поступления товаров на склад, перемещение товаров внутри склада, а также отгрузку товаров клиентам. Каждое движение товара должно быть зарегистрировано и отражено в системе учета для обеспечения точности и надежности данных.

Регистрация заказов от клиентов также является важным процессом в учете товаров на складе. При поступлении заказа от клиента, его данные вводятся в систему учета, где создается соответствующая запись. Затем заказ передается на склад для сборки и отгрузки. Важно, чтобы вся информация о заказе была точно зарегистрирована, чтобы избежать ошибок и удовлетворить потребности клиента.

Организационная структура склада представлена на рисунке 1.1.

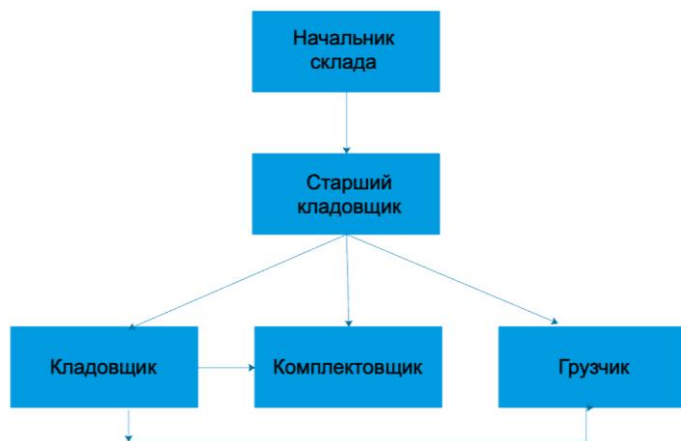


Рисунок 1.1 – Организационная структура склада

1.2 Обзор программных аналогов

В качестве программного аналога рассмотрим 1С: Торговля и склад. В программе имеется:

- наличие функционала для полноценного бухгалтерского, налогового и складского учёта;
- консолидированный учёт в нескольких торговых точках;
- высокая стабильность работы;
- возможность подстройки меню и функционала под конкретного пользователя.

Преимущества программы 1С: Торговля и склад:

- раздельное ведение учёта разных направлений;
- работа с маркированными товарами;
- поддержка операций с валютой;
- автоматизация документооборота;
- ведение налоговой отчётности;
- таможенное сопровождение;
- учёт банковских кредитов.

Недостатки программы:

- софт реализуется через партнёрскую сеть;
- покупка программного обеспечения 1С нерентабельная для микробизнеса;

– в кабинете много функций, которые неактуальны для маленьких предприятий.

Интерфейс программы представлен на рисунке 1.2.

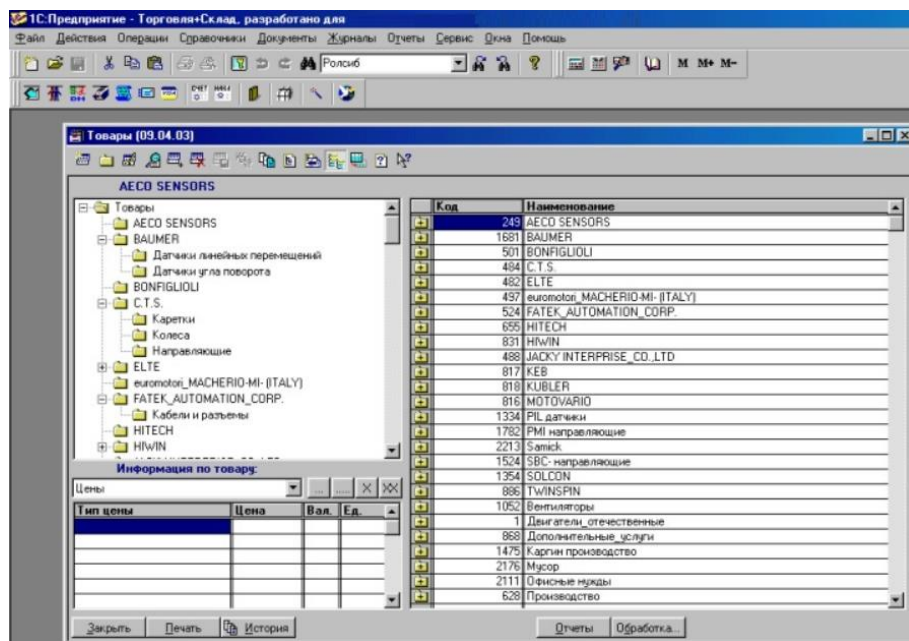


Рисунок 1.2 – Интерфейс программы «1С: Торговля и склад»

Также на рынке имеется программа ЕКАМ. ЕКАМ – это универсальная программа для товарного учета. ПО совместимо с онлайн-кассой по Ф3-54. Софт работает в облачном режиме. Доступ к личному кабинету поддерживается с любого устройства. ЕКАМ подходит для малого или среднего бизнеса. Софт могут использовать владельцы офлайн магазинов, сервисов, кафе и ресторанов, интернет-магазинов.

Преимущества программы:

- интеграция с контрольно-кассовой техникой и торговым оборудованием: электронными весами, сканерами штрих-кодов;
- поддержка 5 движков для интернет-магазинов;
- интеграция с системой маркировки “Честный знак”. При продаже маркированных позиций система передаёт информацию в ОФД;

– генерация технологических карт для промышленных предприятий и заведений общепита (ресторанов, кафе, баров);

– мониторинг экономических показателей: оборота, выручки и прибыли бизнеса;

– автоматическая корректировка розничных цен после исправления закупочных цен;

– возможность ведения программ лояльности.

Недостатки программы:

– нет синхронизации с популярными интернет-магазинами;

– нет API для синхронизации товаров.

Интерфейс программы «ЕКАМ» представлен на рисунке 1.3.

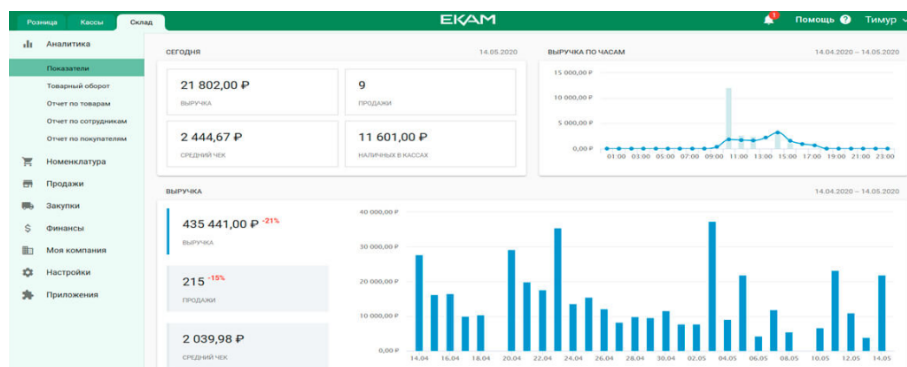


Рисунок 1.3 – Интерфейс программы «ЕКАМ»

Учет товаров на складе и регистрация заказов являются важными процессами для эффективной работы компаний, занимающихся продажей товаров. Для автоматизации этих процессов одними из примеров решений являются 1С: Торговля и склад и ЕКАМ. Обе программы имеют свои преимущества и недостатки.

2 ПОСТАНОВКА ЗАДАЧИ НА РАЗРАБОТКУ ПРОГРАММНОГО СРЕДСТВА ПО ВЕДЕНИЮ СКЛАДСКОГО УЧЁТА И РЕГИСТРАЦИИ ЗАКАЗОВ И ОБЗОР МЕТОДОВ ЕЁ РЕШЕНИЯ

Как уже говорилось выше необходимо разработать комплекс программ для информационной системы "Склад", предназначенной для автоматизации работы сотрудников склада и предоставления им доступа к необходимой информации, а также для выдачи запрашиваемой информации покупателям.

Программное средство должно состоять из следующих частей:

- приложение для администратора;
- приложение для сотрудников склада;
- приложение для покупателей, которые хотят сделать заказ.

Информационная система для сотрудников должна обеспечивать следующий функционал:

- добавление, удаление и редактирование данных о товарах;
- мониторинг заказов, включая их редактирование и добавление в любое время;
- сортировка, фильтрация, формирование отчётов обо всех товарах и заказах;
- подтверждение заказа пользователя;
- возможность управления своим аккаунтом.

Отличие информационной системы администратора от информационной системы сотрудника в том, что администратор имеет возможность добавлять, удалять и редактировать зарегистрированных пользователей, а также получать информацию о сотрудниках.

Информационная система для покупателей должна обеспечивать следующий функционал:

- выбор товара;
- просмотр возможных товаров к заказу;
- просмотр заказов, как действующих, так и завершённых;
- возможность управления своим аккаунтом.

Значительным преимуществом автоматизации данных операций является скорость их выполнения, возможность в любой момент осуществить запрос к данным, находящимся в базе и получить по этому запросу исчерпывающую информацию. Программа позволяет отказаться от необходимости ведения бумажных архивов. В ходе её использования

снижается вероятность введения ошибочной информации. Пользователь программы должен получить от клиента ряд сведений, необходимых для оформления заказа. Для решения такой задачи будет очень полезным использование современных информационных технологий, а именно разработка автоматизированного системного приложения с клиент-серверной архитектурой.

3 ОПИСАНИЕ ПРОЦЕССА ВЕДЕНИЯ УЧЁТА ТОВАРА НА СКЛАДЕ

IDEF0 – методология функционального моделирования и графическая нотация, предназначенная для формализации и описания бизнес-процессов. Отличительной особенностью IDEF0 является ее акцент на соподчиненность объектов. В IDEF0 рассматриваются логические отношения между работами, а не их временная последовательность.

На рисунке 3.1 представлена контекстная диаграмма процесса «Произвести учёт товара».

Входными данными является товар, информация о товаре и заявка покупателей. Данные об объектах обрабатываются согласно уставу магазина и Законодательству РБ. Ресурсами, необходимыми для исполнения процесса, являются программное обеспечение и персонал. Целью данной бизнес-модели является информация о продаже, информация о списании, проданный товар, возвращённый товар и отчёты.

Добавлено примечание ([AK1]): Рисунок 3.1. Глава то третья.

Добавлено примечание ([AK2]): По какому законодательству?

Добавлено примечание ([AK3]): Распишите Вашу контекстную диаграмму подробнее.

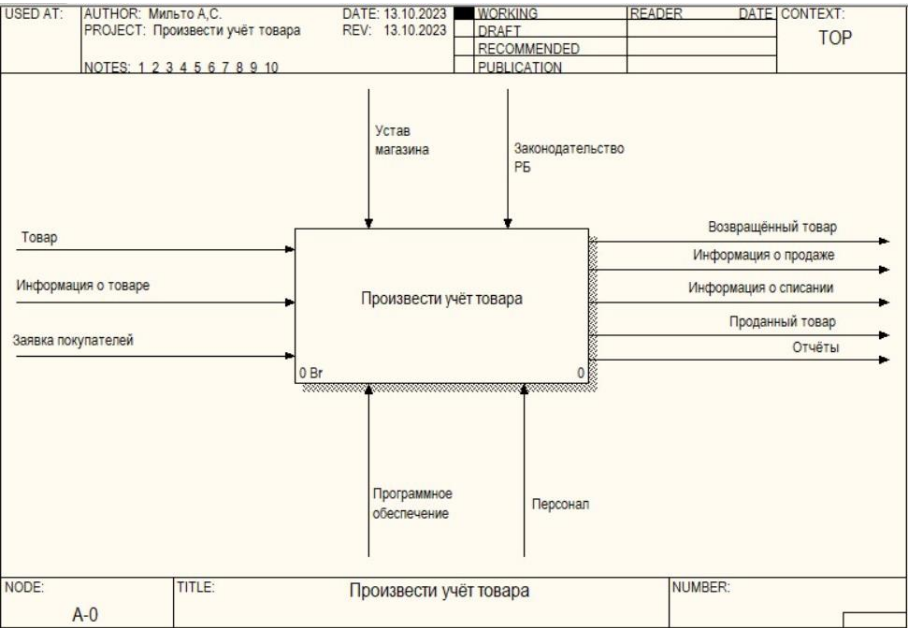


Рисунок 3.1 – Контекстная диаграмма «Произвести учёт товара»

На рисунке 3.2 представлена диаграмма декомпозиции процесса «Произвести учёт товара». На данной декомпозиции представлены такие процессы, как «Получить информацию со склада», «Получить информацию из зала» и «Сформировать отчёт».

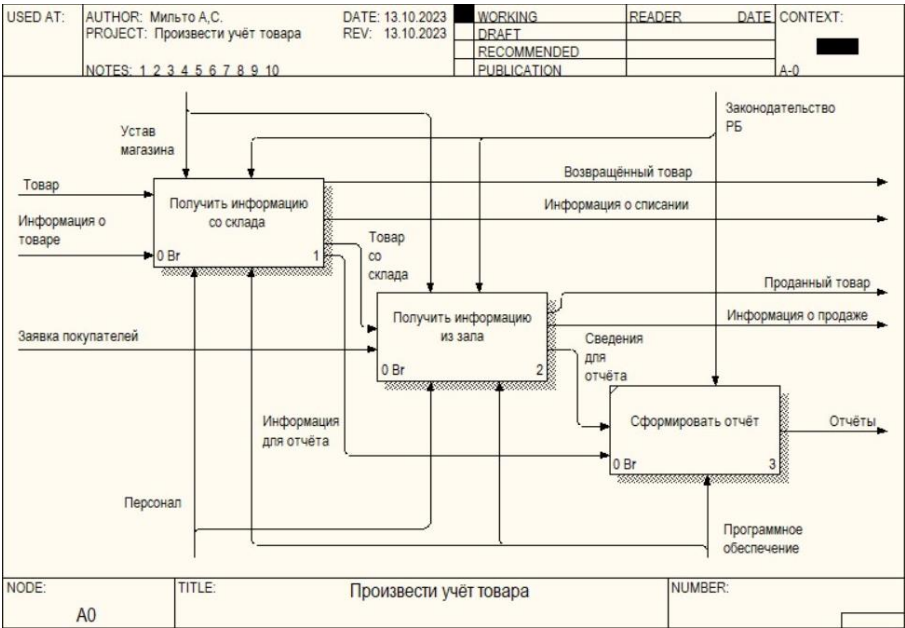


Рисунок 3.2 – Диаграмма декомпозиции процесса «Произвести учёт товара»

На рисунке 3.3 представлена диаграмма декомпозиции процесса обработка заказа. На данной декомпозиции представлены такие процессы, как «Принять товар на склад», «Оформить приход товара», «Проверить качество товара» и «Распределить товар на склад».

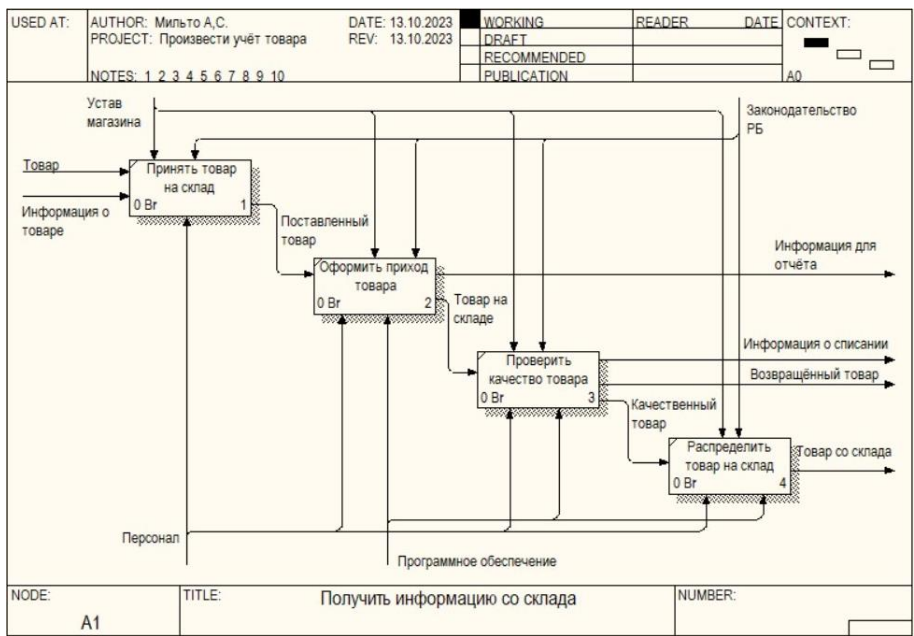


Рисунок 3.3 – Диаграмма декомпозиции процесса «Получить информацию со склада»

На рисунке 3.4 представлена диаграмма декомпозиции процесса «Получить информацию из зала». На данной диаграмме представлены такие процессы, как «Распределить товар», «Выбрать товар», «Продать товар» и «Выдать товар».

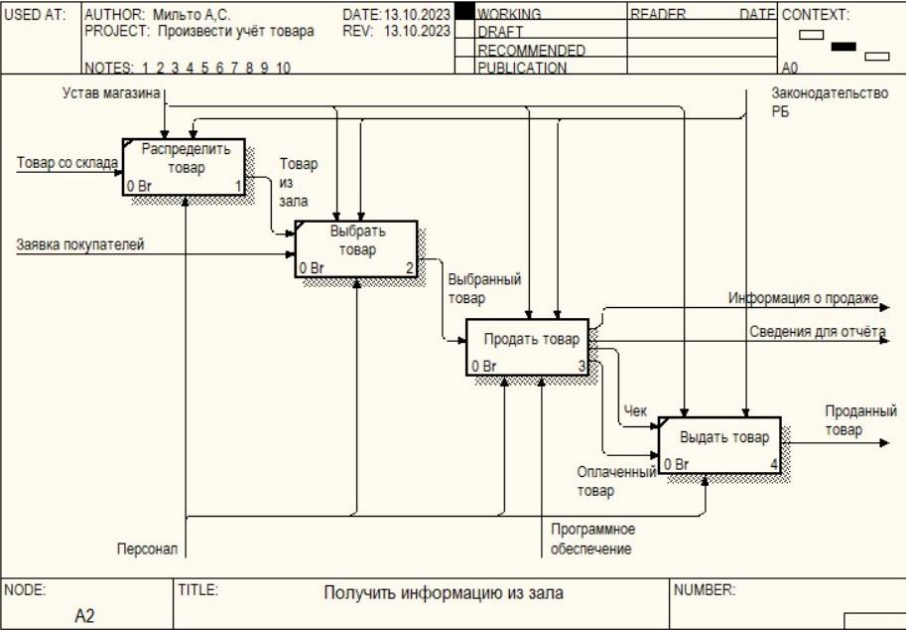


Рисунок 3.4 – Диаграмма декомпозиции процесса «Получить информацию из зала»

На рисунке 3.5 представлена диаграмма декомпозиции процесса «Продать товар». На данной диаграмме представлены такие процессы, как «Оформить продажу товара» и «Принять оплату».

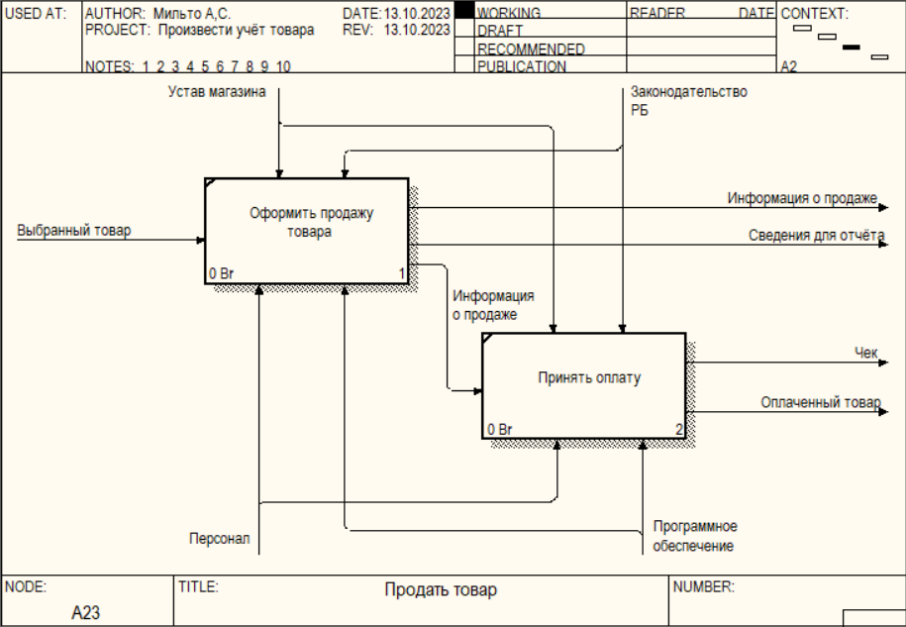


Рисунок 3.5 – Диаграмма декомпозиции процесса «Продать товар»

Таким образом, из представленных декомпозиций можно сделать вывод о том, какой сложно является работа склада. Не возникает сомнений, что такая работа нуждается в автоматизации некоторых процессов для повышения качества обслуживания заказчиков.

4 ИНФОРМАЦИОННАЯ МОДЕЛЬ СИСТЕМЫ И ЕЕ ОПИСАНИЕ

Сущностями данной модели являются suppliers, customers, orders, products, productsgroups, completedorders и users.

Сущность «suppliers» содержит информацию о поставщике: id, имя (название организации), страна и номер телефона.

Сущность «customers» содержит информацию о заказчике: id, имя, страна и адрес.

Сущность «orders» содержит информацию о заказах: id, id товара, количество, дата продажи стоимость и id заказчика.

Сущность «products» содержит информацию о товарах: id, id поставщика, id группы товаров, наименование, цена, количество и дата поступления.

Сущность «productsgroups» содержит информацию о группе товаров: id и наименование группы товаров.

Сущность «user» содержит информацию о зарегистрированных пользователях: id, логин, пароль и роль.

Сущность «completedorders» содержит информацию о выполненных заказах: id группы товаров, количество, дата заказа, дата подтверждения, общая стоимость заказа и id покупателя.

Общая схема связанных сущностей базы данных представлена на рисунке 4.1.

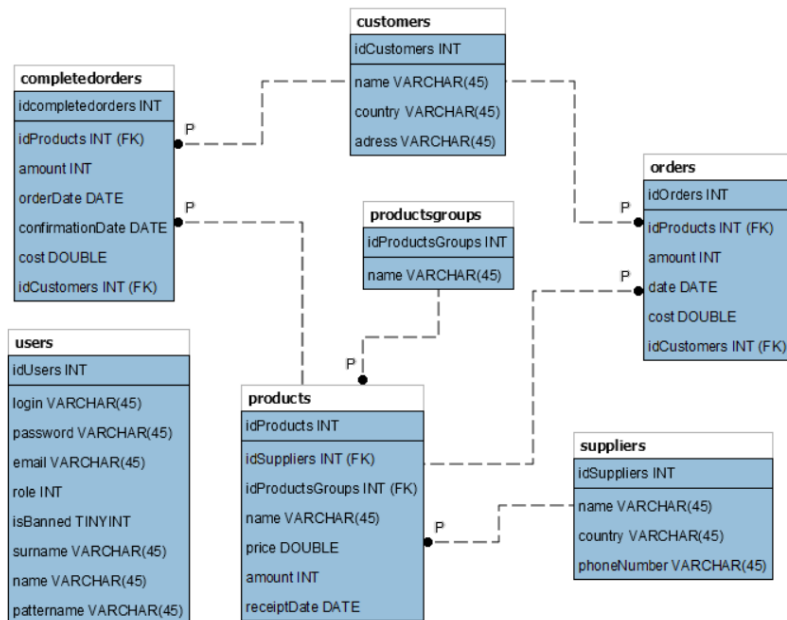


Рисунок 4.1 – Информационная модель базы данных

Для доказательства нахождения таблиц в третьей нормальной форме необходимо воспользоваться следующими понятиями. Таблица находится в первой нормальной форме, если все её поля имеют простые значения. Таблица находится во второй нормальной форме, если она находится в первой нормальной форме, а каждое не ключевое поле функционально полно зависит от составного ключа. Таблица находится в третьей нормальной форме, если она находится во второй нормальной форме, и каждое не ключевое поле не транзитивно зависит от первичного ключа.

Таким образом, база данных приведена к третьей нормальной форме, поскольку у каждой таблицы имеется всего один первичный ключ, а каждое не ключевое поле не транзитивно зависит от первичного ключа, т.е. изменив значение в одном столбце не потребуется изменение в другом столбце.

5 МОДЕЛИ ПРЕДСТАВЛЕНИЯ СИСТЕМЫ УЧЁТА ТОВАРОВ НА СКЛАДЕ НА ОСНОВЕ UML

UML – язык графического описания для объектного моделирования в области разработки программного обеспечения, моделирования бизнес-процессов, системного проектирования и отображения организационных структур.

5.1 Спецификация вариантов использования системы

Вариант использования описывает типичное взаимодействие между пользователем и системой. В простейшем случае вариант использования определяется в процессе обсуждения с пользователем тех функций, которые он хотел бы реализовать. Диаграмма вариантов использования представлена на рисунке 5.1.



Рисунок 5.1 – Диаграмма вариантов использования

Существует три актера: администратор, работник склада и пользователь(покупатель). Заказчик может просмотреть товары, заказать их, изменить собственные данные аккаунта. Администратор и работник склада имеют доступ к функциям склада: просмотр товаров, CRUD товаров, просмотр заказов, их подтверждение, поиск информации по различным критериям, создание прайс-листа, формирование отчёта о проданных товарах за период. Администратор от работника склада отличается возможностью работы с аккаунтами пользователей. Он может добавлять новых пользователей, изменять данные уже созданных аккаунтов, блокировать пользователей, а также удалять их.

5.2 Описание диаграммы последовательности системы

Диаграмма последовательности описывает поведение только одного варианта использования. На такой диаграмме отображаются только экземпляры объектов и сообщения, которыми они обмениваются между собой.

На рисунке 5.2 представлена диаграмма деятельности авторизации пользователя.

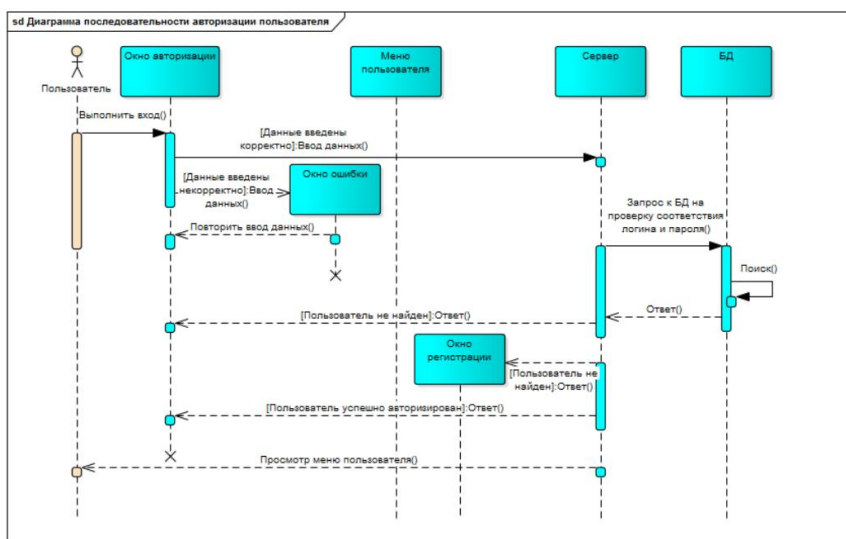


Рисунок 5.2 – Диаграмма последовательности авторизации пользователя

На данной диаграмме пользователь выполняет вход в программу, вводит данные (логин и пароль) в окне авторизации.

В случае корректно введенных данных сервер посылает заброс к БД на проверку соответствия логина и пароля, т. е. введенные логин и пароль пользователя сравнивается со всеми имеющимися в БД логинами и паролями пользователей. Если соответствие найдено, сервер информирует пользователя об успешной авторизации. При этом жизненный цикл окна авторизации прекращается, а пользователю предоставляется меню для последующей работы с системой.

Если данные были введены некорректно, появляется окно ошибки, информирующее о неправильно введенных данных.

Если в БД не было найдено соответствие логину и паролю, которые ввел пользователь, сервер информирует пользователя об отсутствии такого пользователя.

5.3 Описание диаграммы состояний

Диаграмма состояний описывает все возможные состояния одного экземпляра определенного класса и возможные последовательности его переходов из одного состояния в другое, то есть моделирует все изменения состояний объекта как его реакцию на внешние воздействия.

На рисунке 5.3 представлена диаграмма состояний заказа товаров.

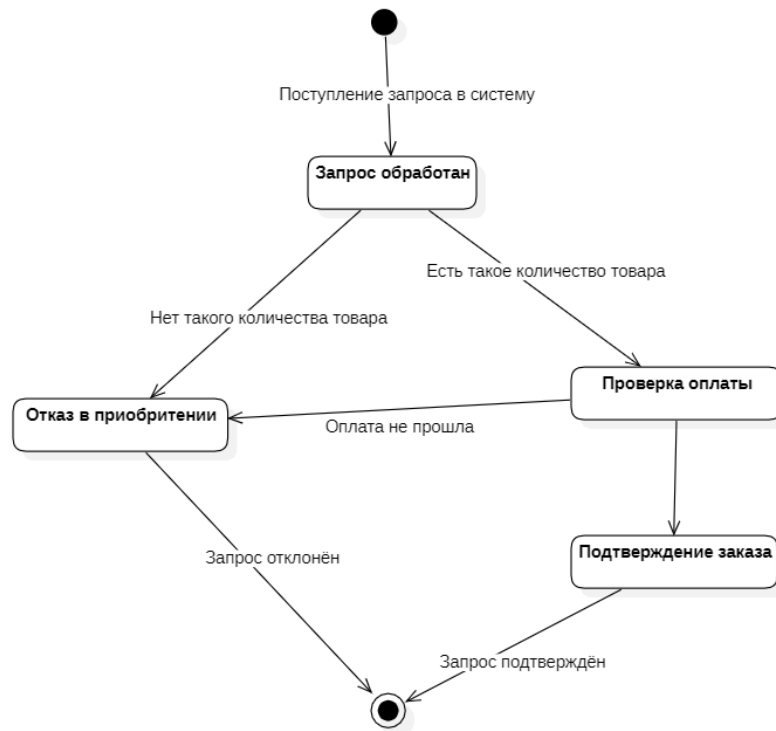


Рисунок 5.3 – Диаграмма состояний

5.4 Описание диаграммы компонентов

На языке унифицированного моделирования диаграмма компонентов показывает, как компоненты соединяются вместе для формирования более крупных компонентов или программных систем. Она иллюстрирует архитектуры компонентов программного обеспечения и зависимости между ними. Другими словами, диаграмма компонентов дает упрощенное представление о сложной системе, разбивая ее на более мелкие компоненты. Каждый из элементов показан в прямоугольной рамке с названием, написанным внутри. Соединители определяют отношения и зависимости между различными компонентами. Среди основных целей диаграммы компонентов можно выделить: – визуализация общей структуры исходного кода программы; – обеспечение многократного использования фрагментов кода; – представление концептуальной и физической базы данных. Диаграмма компонентов представлена на рисунке 5.4. Она позволяет определить состав компонентов программы, а также установить зависимость между ними.

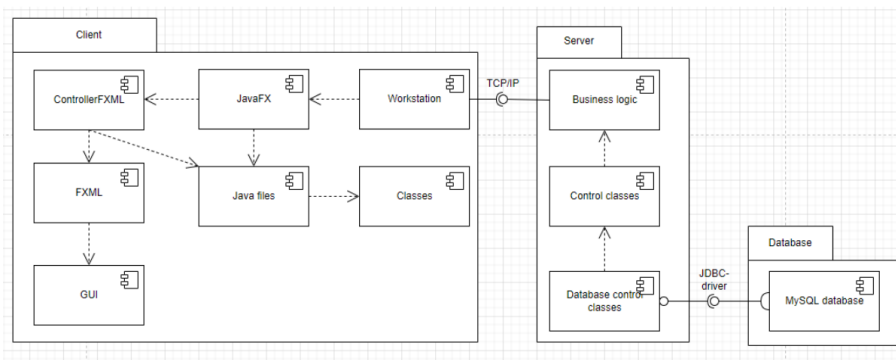


Рисунок 5.4 – Диаграмма компонентов

5.5 Описание диаграммы развертывания

Диаграмма развертывания предназначена для представления физического расположения системы, показывая, на каком физическом оборудовании запускается та или иная составляющая программного обеспечения. На рисунке 5.5 представлена диаграмма развёртывания.

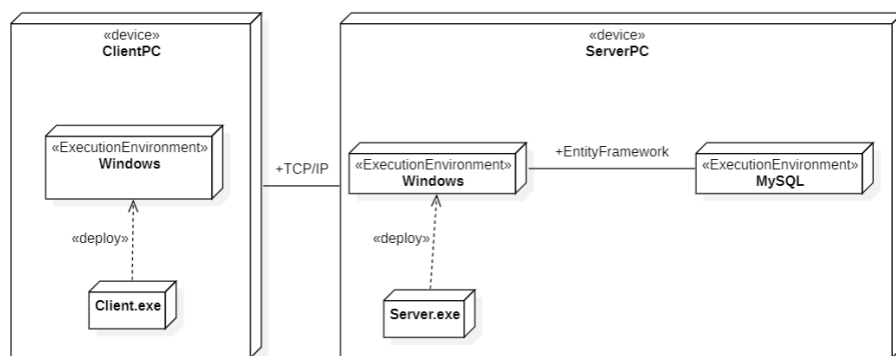


Рисунок 5.5 – Диаграмма развёртывания

5.6 Описание диаграммы классов

На рисунке 5.6 представлена диаграмма классов серверной части приложения.



Рисунок 5.6 – Диаграмма классов

После составления UML диаграмм можно сказать, что появляется наглядное представление различных аспектов системы, таких как ее структура, поведение и взаимодействие между компонентами. Это поможет участникам проекта лучше понять систему, обнаружить возможные проблемы и улучшить процесс проектирования и разработки. Также UML диаграммы могут служить важным инструментом для общения между участниками проекта и заинтересованными сторонами.

6 ОБОСНОВАНИЕ ВЫБОРА ПРОГРАММНЫХ СРЕДСТВ РАЗРАБОТКИ

В рамках курсового проекта было принято решение разработать автоматизированную систему ведения учёта товаров на складе и регистрации заказов. Учитывая возможности, имеющегося оборудования и программного обеспечения, необходимо создать современный программный продукт, избегая таких недостатков существующих коммерческих предложений, как высокая стоимость внедрения и сопровождения и слабая ориентированность на пользователя с разной профессиональной подготовкой. Также необходимо уделить особое внимание надежности приложения и простоте его интерфейса.

Поэтому для разработки программы ведения складского учёта и регистрации заказов были выбраны:

1 СУБД MySQL – предоставляет мощные средства для доступа, настройки, администрирования, разработки всех компонентов базы данных и управления ими. MySQL – это реляционная система управления базами данных. То есть данные в ее базах хранятся в виде логически связанных между собой таблиц, доступ к которым осуществляется с помощью языка запросов SQL. MySQL – свободно распространяемая система. Кроме того, это достаточно быстрая, надежная и, главное, простая в использовании СУБД.

2 Среда IntelliJ IDEA 2023 и язык программирования Java.

IntelliJ IDEA – это интегрированная среда разработки (IDE) для языков программирования Java, Kotlin, Groovy и Scala. Ее основное преимущество заключается в том, что она предоставляет разработчикам широкий набор инструментов и функций для улучшения процесса разработки, увеличения производительности и улучшения качества кода.

Некоторые основные преимущества IntelliJ IDEA:

- умный редактор кода. IntelliJ IDEA обладает умным редактором кода, который предлагает автозавершение кода, быструю навигацию по проекту, рефакторинг и другие инструменты для повышения производительности разработчика;

- интеграция с другими инструментами. IntelliJ IDEA интегрируется с такими инструментами, как системы контроля версий (например, Git), средства сборки (например, Maven или Gradle) и серверы приложений, что делает процесс разработки более удобным и эффективным;

- анализ кода. Среда разработки предоставляет возможности для анализа кода, выявления потенциальных проблем и предложения способов их исправления;

- поддержка многих языков. IntelliJ IDEA поддерживает не только Java, но также Kotlin, Groovy и Scala, что делает ее универсальным инструментом для разработки на различных языках программирования;

- поддержка различных плагинов. Среда разработки позволяет установить различные плагины для расширения ее функциональности в соответствии с потребностями разработчика.

Данный курсовой проект написан на объектно-ориентированном языке Java. Java имеет ряд преимуществ над другими языками, такие как:

- переносимость. Java является платформенно-независимым языком, что означает, что программы, написанные на Java, могут быть запущены на любой платформе, поддерживающей Java Virtual Machine (JVM);

- безопасность. Java обеспечивает высокий уровень безопасности благодаря своей архитектуре и механизмам безопасности, таким как проверка типов, управление памятью и отсутствие указателей;

- простота использования. Java обладает простым и интуитивно понятным синтаксисом, что упрощает процесс разработки программ;

- масштабируемость. Java поддерживает различные уровни масштабирования, начиная от небольших приложений до крупных корпоративных систем;

- обширная библиотека. Java имеет обширную стандартную библиотеку классов, которая предоставляет разработчикам множество готовых решений для различных задач;

- поддержка многопоточности. Java имеет встроенную поддержку многопоточности, что позволяет создавать эффективные и отзывчивые приложения;

- поддержка сетевых технологий. Java обладает мощными средствами для работы с сетевыми технологиями, такими как создание клиент-серверных приложений и веб-приложений;

- поддержка разработки мобильных приложений. С появлением платформы Android, Java стал одним из основных языков программирования для разработки мобильных приложений.

В курсовом проекте использовались такие паттерны проектирования, как DAO и MVC.

DAO (Data Access Object) – это паттерн проектирования, который используется для управления доступом к данным в приложении. Он предоставляет абстрактный интерфейс к данным, скрывая детали их хранения и доступа. DAO паттерн позволяет разделить бизнес-логику приложения от

кода доступа к данным, что делает приложение более гибким и легко поддающимся изменениям.

Основная цель DAO паттерна – это разделение ответственностей между бизнес-логикой и доступом к данным. Он позволяет создать отдельные классы или компоненты, которые отвечают только за доступ к определенным типам данных (например, пользователям, товарам, заказам и т.д.), и предоставляют методы для чтения, записи, обновления и удаления этих данных.

MVC (Model-View-Controller) – это паттерн проектирования, который широко используется в разработке программного обеспечения для построения пользовательского интерфейса. Он разделяет приложение на три основных компонента: Модель, Представление и Контроллер.

Модель представляет собой структуру данных приложения и бизнес-логику, которая управляет этими данными. Она отвечает за хранение, обработку и манипуляцию данными, а также за взаимодействие с базой данных или другими источниками данных.

Представление отвечает за отображение данных пользователю и взаимодействие с ним. Это может быть пользовательский интерфейс, веб-страница, отчет или любой другой способ отображения информации. Представление получает данные из модели и отображает их пользователю, а также отправляет пользовательский ввод контроллеру.

Контроллер является посредником между моделью и представлением. Он обрабатывает пользовательский ввод, взаимодействует с моделью для получения необходимых данных и обновления их, а затем передает эти данные представлению для отображения. Контроллер также отвечает за обработку пользовательских действий, таких как нажатия кнопок, ввод текста и т.д.

Использование паттерна MVC позволяет разделить бизнес-логику, пользовательский интерфейс и управление данными на отдельные компоненты, что делает приложение более гибким, легко поддающимся изменениям и повторному использованию кода. Кроме того, он способствует улучшению поддерживаемости и расширяемости приложения.

Таким образом, выбор среды разработки IntelliJ IDEA и языка программирования Java для создания программы ведения складского учета и регистрации заказов обоснован. IntelliJ IDEA предоставляет широкий набор инструментов для улучшения процесса разработки, увеличения производительности и улучшения качества кода, а Java обладает рядом преимуществ, таких как переносимость, безопасность, простота использования, масштабируемость и поддержка многопоточности. Кроме того, для хранения данных складского учета и заказов, MySQL является отличным выбором как реляционная база данных. Он обеспечивает надежное

хранение и управление данными, поддерживает высокую производительность и масштабируемость, а также имеет широкую поддержку в сообществе разработчиков. Таким образом, использование MySQL в сочетании с IntelliJ IDEA и Java обеспечивает надежную и эффективную основу для создания программы ведения складского учета и регистрации заказов. Использование паттернов проектирования DAO и MVC в курсовом проекте позволило создать структурированное, модульное и гибкое приложение. Паттерн MVC помог разделить бизнес-логику, пользовательский интерфейс и управление данными, что упростило разработку и поддержку кода. Паттерн DAO позволил отделить доступ к данным от бизнес-логики, что сделало приложение более гибким и легко поддающимся изменениям.

7 ОПИСАНИЕ АЛГОРИТМОВ, РЕАЛИЗУЮЩИХ БИЗНЕС-ЛОГИКУ СИСТЕМЫ

7.1 Схема алгоритма клиент-серверного взаимодействия

На рисунке 7.1 представлена схема алгоритма клиент-серверного взаимодействия.



Добавлено примечание ([CM4]):

Добавлено примечание ([CM5]): Создание сервера, чтарт , отправка как обработка.
В старте цикла блока название и условие работы в скобках, когда закрываем только название.

Рисунок 7.1 – Схема алгоритма клиент-серверного взаимодействия

7.2 Схема алгоритма работы администратора

Данный алгоритм позволяет выполнять администратору все необходимые ему функции: просматривать текущие товары на складе, удалять, корректировать и добавлять товары, просматривать списки пользователей, поставщиков, заказов, а также добавлять информацию о поставщиках, просматривать статистику заказов.

Блок-схема данного алгоритма представлена на рисунке 7.2.



Рисунок 7.2 – Схема алгоритма работы администратора

7.3 Схема алгоритма добавления информации о товаре

Данный алгоритм позволяет добавлять информацию о товарах в базу. Для добавления информации необходимо ввести данные, создать запрос на проверку существования добавляемого товара с уже существующими. Если информацию уже существует в базе данных, то количество товара увеличивается на соответствующее значение. Если же информация не существует в базе, то она добавляется.

Блок-схема алгоритма представлена на рисунке 7.3.

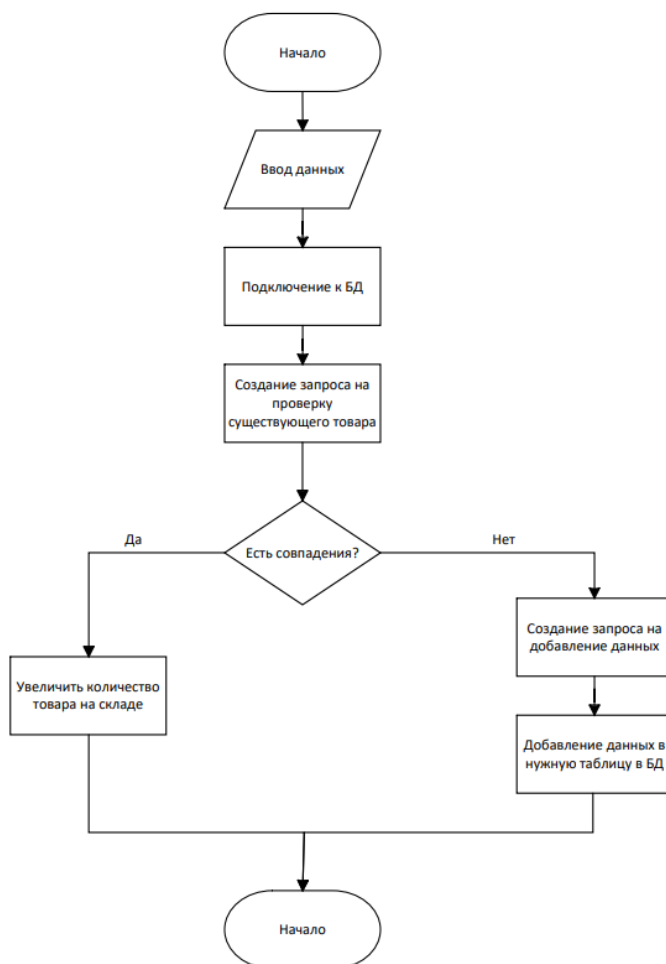


Рисунок 7.3 – Схема алгоритма добавления информации о товаре

Добавлено примечание ([АК6]): 1.ГОСТ
2.Ветвление подписать
3.Не все диаграммы.
4.На use case диаграмме ничего не видно.

8 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

С программой можно работать от лица администратора, работника склада и покупателя. От того, от лица кого вы авторизовались зависит перечень возможных действий. Так, при авторизации как покупатель, Вы можете только создать заказ, редактировать свой профиль и просматривать список заказов, при авторизации как работник склада Вы сможете добавлять новые товары, редактировать их, просматривать данные, создавать отчёты, в то время как администратор может не только просматривать и изменять данные, но и работать с аккаунтами пользователей.

При запуске программы появляется меню авторизации, представленное на рисунке 8.1.

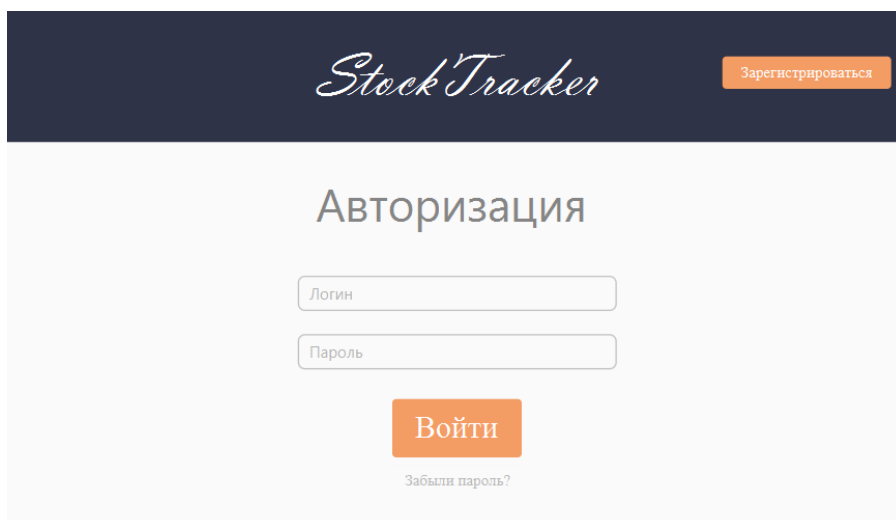
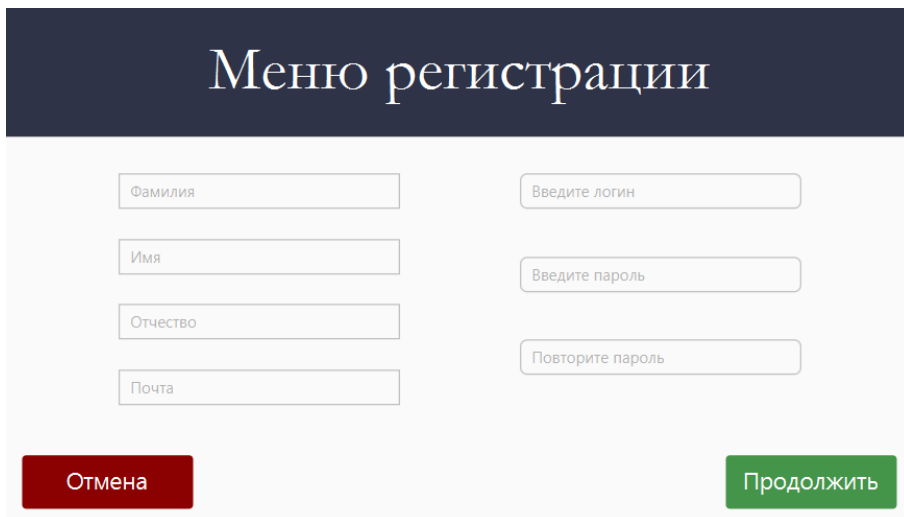


Рисунок 8.1 – Меню авторизации

При нажатии на кнопку «Зарегистрироваться» появляется меню регистрации, представленное на рисунке 8.2.



Меню регистрации

Фамилия

Имя

Отчество

Почта

Введите логин

Введите пароль

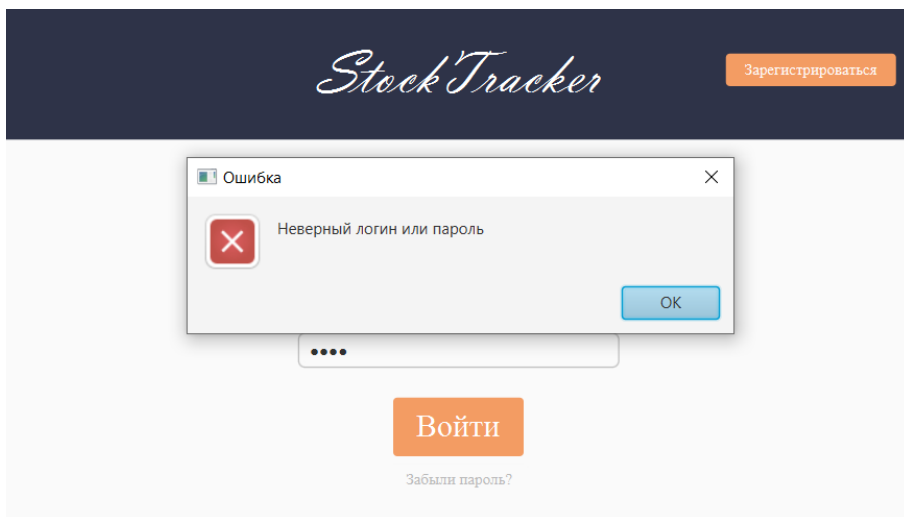
Повторите пароль

Отмена

Продолжить

Рисунок 8.2 – Меню регистрации

При вводе неверного логина или пароля в меню авторизации появляется окно, представленное на рисунке 8.3, информирующее об ошибке.



Stock Tracker

Зарегистрироваться

Ошибка

Неверный логин или пароль

OK

Войти

Забыли пароль?

Рисунок 8.3 – Информирование об ошибке авторизации

При авторизации в роли администратора доступна работа с аккаунтами пользователями и работа со складом. Интерфейс представлен на рисунке 8.4.



Рисунок 8.4 – Главное меню администратора

При входе в аккаунт как пользователь доступны такие операции, как формирование заказа, просмотр как действующих заказы, так и уже выполненных, а также редактирование собственного аккаунта. Пример интерфейса представлен на рисунке 8.5.

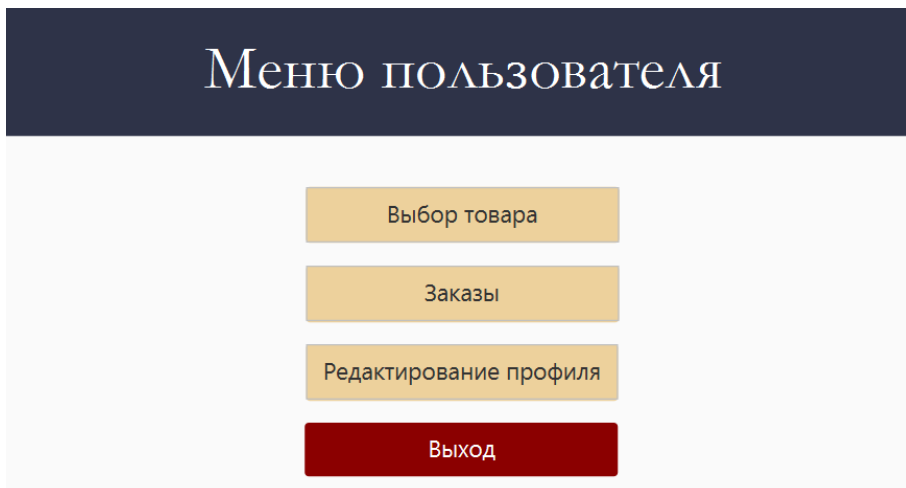


Рисунок 8.5 – Главное меню пользователя

Работник склада от администратора отличается только тем, что он не имеет возможности работы с аккаунтами других пользователей, работа на складе полностью идентична работе администратора. Пример главного меню работника склада представлен на рисунке 8.6.

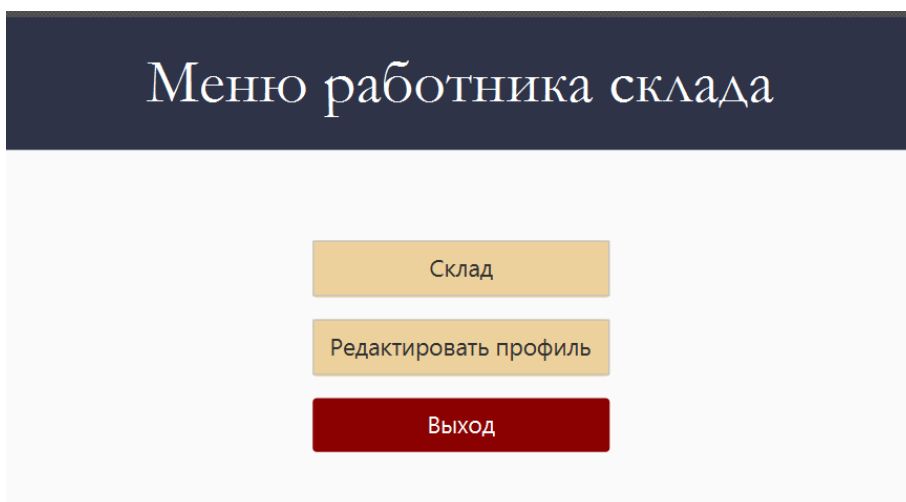


Рисунок 8.6 – Главное меню работника склада

Меню работы с аккаунтами пользователя доступно только администратору. В таблицу выводятся все созданные аккаунты, доступны функции создания нового аккаунта, блокировка, удаление и редактирование. Меню работы с аккаунтами представлено на рисунке 8.7.

Назад Аккаунты Add Block Edit Delete							
Логин	Пароль	Почта	Роль	Заблокирован	Фамилия	Имя	Отчество
Stepa	Stepa	Stepa	user	false	Stepa	Stepa	Stepa
asd	asd	asd	admin	false	asd	asd	asd
sashamilto	sashamilto	sashamilto3@...	user	false	Мильто	Александр	Сергеевич

Рисунок 8.7 – Меню работы с аккаунтами пользователей

Для вызова меню редактирования аккаунта следует выбрать нужный аккаунт и нажать на кнопку «Add». Появятся поля аккаунта, в которые автоматически будет записано существующее значение и его можно изменить. Это же меню для редактирования собственного аккаунта доступно и для других ролей. Меню редактирования представлено на рисунке 8.8.

Меню редактирования

Мильто

Александр

Сергеевич

sashamilito3@gmail.com

sashamilito

.....

user

Отмена

Подтвердить

Рисунок 8.8 – Меню редактирования аккаунта пользователя

При работе от имени пользователя и работника склада меню редактирования собственного аккаунта отличается от меню редактирования администратора только тем, что пользователь не может выбрать себе роль. Пример меню представлен на рисунке 8.9.

Меню редактирования

Stepa1

Stepa

Stepa

Stepa

Stepa

Stepa

Stepa

.....

Назад

Продолжить

Рисунок 8.9 – Меню редактирования собственного аккаунта

При работе со складом доступны следующие функции: добавление информации, товары на складе, заказы и отчёты. Меню склада представлено на рисунке 8.10.

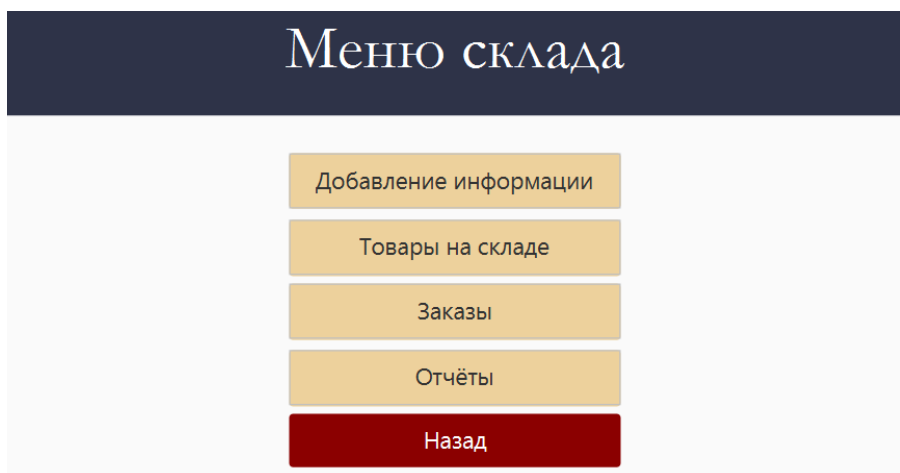


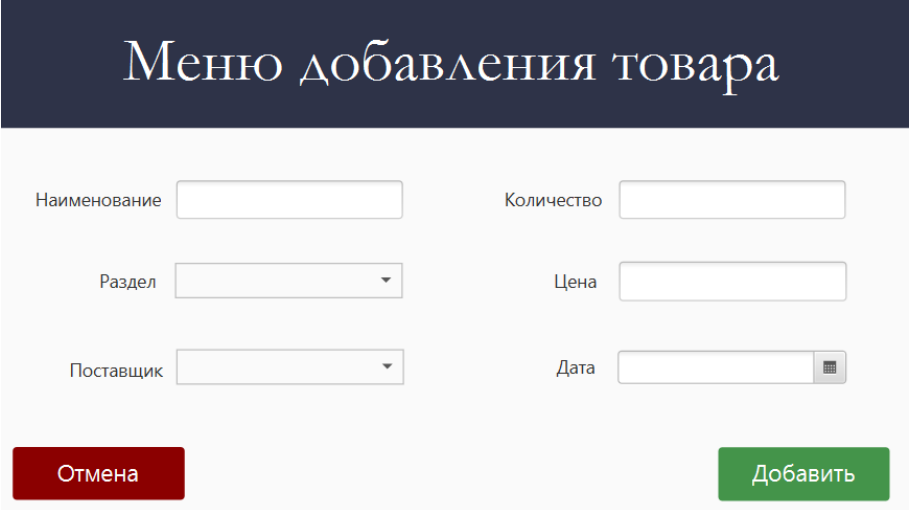
Рисунок 8.10 – Меню склада

Добавить можно товар, группу товаров и поставщика. Меню добавления представлено на рисунке 8.11.



Рисунок 8.11 – Меню добавления

При добавлении товара необходимо ввести наименование, выбрать раздел и поставщика из выпадающего списка, количество, цену и дату завоза товара. Меню представлено на рисунке 8.12.



The screenshot shows a web form titled "Меню добавления товара" (Menu for adding goods) in a dark blue header. The form is set against a light gray background and contains several input fields arranged in two columns. The left column includes fields for "Наименование" (Name), "Раздел" (Section) with a dropdown arrow, and "Поставщик" (Supplier) with a dropdown arrow. The right column includes fields for "Количество" (Quantity), "Цена" (Price), and "Дата" (Date) with a calendar icon. At the bottom left is a red button labeled "Отмена" (Cancel), and at the bottom right is a green button labeled "Добавить" (Add).

Меню добавления товара	
Наименование	Количество
Раздел	Цена
Поставщик	Дата
Отмена	Добавить

Рисунок 8.12 – Меню добавления товара

Меню просмотра товара представлено на рисунке 8.13. Для просмотра товаров нужно выбрать раздел. В этом меню доступны функции редактирования и удаления товара. Для этого необходимо сначала выбрать необходимый товар, а потом нажать соответствующую кнопку.

Назад Список товаров Компьютеры Edit Delete					
Поставщик	Группа товаров	Наименование	Стоимость	Количество	Дата прихода
Lenovo	Компьютеры	Lenovo Legion 3	5.0	4	Mon Dec 11 00:00:00 MSK ...
Lenovo	Компьютеры	IdeaPad Pro 5i	2352.0	4	Thu Dec 07 00:00:00 MSK 2...
Lenovo	Компьютеры	IdeaPad Slim 3i	23529.0	346	Thu Dec 07 00:00:00 MSK 2...

Рисунок 8.13 – Меню просмотра товаров на складе

При редактировании товара в поля подгружаются значения соответствующих характеристик товара, и они доступны для редактирования. Меню представлено на рисунке 8.14.

Меню редактирования товара			
Наименование	IdeaPad Pro 5i	Количество	4
Раздел	Компьютеры	Цена	2352.0
Поставщик	Lenovo	Дата	07.12.2023
Отмена		Изменить	

Рисунок 8.14 – Меню редактирования товара

Пользователь может создать заказ. Для этого необходимо выбрать товар из списка и нажать на кнопку «Заказать». Пример меню для заказа представлен на рисунке 8.15.

Назад

Компьютеры

Заказать

Наименование	Стоимость	Количество
Lenovo Legion 3	5.0	4
IdeaPad Pro 5i	2352.0	4
IdeaPad Slim 3i	23529.0	346

Рисунок 8.15 – Меню создания заказа

Для завершения создания заказа пользователю необходимо ввести нужное количество, наименование организации, страну и свой адрес. Пример меню подтверждения создания представлен на рисунке 8.16.

Меню подтверждения заказа

Отмена

Подтвердить

Рисунок 8.16 – Меню подтверждения создания заказа

Администратор и работник склада могут подтвердить заказ в соответствующем пункте меню. При подтверждении заказ переносится в таблицу завершённых заказов и количество товара на складе уменьшается на соответствующее количество. Меню действующих заказов, которые необходимо подтвердить, представлено на рисунке 8.17.

Меню действующих заказов

Поиск

Подтвердить

Назад

Наименование товара	Количество	Стоимость	Дата заказа	Заказчик
IdeaPad Slim 3i	5	117645.0	2023-12-11	dsgs

Рисунок 8.17 – Меню подтверждения заказа

В меню просмотра завершённых заказов выводятся заказы, подтверждённые администратором или работником склада. Меню завершённых заказов представлено на рисунке 8.18.

с

по

Поиск

Назад

Наименование товара	Количество	Стоимость	Дата заказа	Дата подтверждения	Заказчик
fdgdfg	3	5.0	2023-12-09	2023-12-09	dsgsd
fdgdfg	4	5.0	2023-12-09	2023-12-10	Stepasha
fdgdfg	3	5.0	2023-12-09	2023-12-10	Stepa

Рисунок 8.17 – Меню завершённых заказов

Руководство пользователя облегчит процесс знакомство пользователя с приложением, так как в нем описаны способы использования программного средства.

9 РЕЗУЛЬТАТ ТЕСТИРОВАНИЯ СИСТЕМЫ УЧЕТА ТОВАРОВ И РЕГИСТРАЦИИ ЗАКАЗОВ

Тестирование является неотъемлемой частью разработки программных средств, так как пользование нашим приложением должно быть максимально приятным и удобным. Для тестирования в языке Java можно использовать фреймворки Junit и Mockito. Основную бизнес логику приложения больше всего описывает класс `RequestHandler`, поэтому для тестов был выбран именно этот класс. Так как этот класс имеет довольно много зависимостей, то для тестирования был выбран фреймворк Mockito, который позволит тестировать класс, не создавая для него зависимостей. На рисунке 9.1 изображена реализация теста проверки, что вернёт метод обработки запроса при корректно введённых данных.

```
@Test
void Test_Login_with_Valid_Data(){
    User user = new User();
    when(userDAO.FindUserByLogin(user)).thenReturn(user);
    LoginResponse expected = new LoginResponse(user, context: "");
    LoginResponse real = (LoginResponse) requestHandler.HandleRequest(new LoginRequest(user));
    Assertions.assertEquals(expected.getUser(), real.getUser());
}
```

Рисунок 9.1 – Реализация теста при корректно введённых данных логина

На рисунке 9.2 изображена реализация теста проверки, что вернёт метод обработки запроса при некорректно введённых данных логина.

```
void Test_Login_with_Invalid_Data(){
    User user = new User();
    when(userDAO.FindUserByLogin(user)).thenReturn(null);
    LoginResponse expected = new LoginResponse(user: null, context: "Неверный логин или пароль");
    LoginResponse real = (LoginResponse) requestHandler.HandleRequest(new LoginRequest(user));
    Assertions.assertEquals(expected.getContext(), real.getContext());
}
```

Рисунок 9.2 – Реализация теста при некорректно введённых данных логина

На рисунке 9.3 изображена реализация теста, в котором проверяется, что вернёт метод обработки запроса, если при получении списка пользователей произойдёт ошибка.

```

@Test
void Test_GetUsers_When_Error(){
    when(userDAO.GetUsers()).thenReturn( null);
    var real = (GetUsersResponse) requestHandler.HandleRequest(new GetUsersRequest());
    Assertions.assertNull(real.getUsers());
}

```

Рисунок 9.3 – Реализация теста, если произойдёт ошибке при получении списка пользователей

На рисунке 9.4 изображена реализация теста, в котором проверяется, что вернёт метод обработки запроса, если при получении списка пользователей не произойдёт ошибка.

```

@Test
void Test_GetUsers_With_Valid_Data(){
    ArrayList<User> users = new ArrayList<>();
    for(int i = 0; i < 10; i++){
        users.add(new User());
    }
    when(userDAO.GetUsers()).thenReturn(users);
    var real = (GetUsersResponse) requestHandler.HandleRequest(new GetUsersRequest());
    Assertions.assertEquals( expected: 10, real.getUsers().size());
}

```

Рисунок 9.4 – Реализация теста, если не произойдёт ошибка при получении списка пользователей

На рисунке 9.5 изображена реализация теста, в котором проверяется, что вернёт метод обработки запроса, если при получении списка заказов не произойдёт ошибка.

```

@Test
void Test_GetOrders_With_Valid_Data(){
    ArrayList<Order> orders = new ArrayList<>();
    for(int i = 0; i < 10; i++){
        orders.add(new Order());
    }
    when(productDAO.GetOrders()).thenReturn(orders);
    var real = (GetOrdersResponse) requestHandler.HandleRequest(new GetOrdersRequest());
    Assertions.assertEquals( expected: 10, real.getOrders().size());
}

```

Рисунок 9.5 – Реализация теста, если не произойдёт ошибка при получении списка заказов

На рисунке 9.6 изображена реализация теста, в котором проверяется, что вернёт метод обработки запроса, если при получении списка заказов произойдёт ошибка.

```
@Test
void Test_GetOrders_When_Error(){
    when(productDAO.GetOrders()).thenReturn( t null);
    var real = (GetOrdersResponse) requestHandler.HandleRequest(new GetOrdersRequest());
    Assertions.assertNull(real.getOrders());
}
```

Рисунок 9.6 – Реализация теста, если произойдёт ошибке при получении списка заказов

После реализации данных тестов необходимо запустить их и убедиться, что всё работает исправно. На рисунке 9.7 изображён результат запуска тестов.

requestHandlerTest	1 sec 14 ms
Test_Login_with_Valid_Data()	982 ms
Test_Login_with_Invalid_Data()	10 ms
Test_GetUsers_When_Error()	7 ms
Test_GetUsers_With_Valid_Data()	4 ms
Test_GetOrders_When_Error()	6 ms
Test_GetOrders_With_Valid_Data()	5 ms

Рисунок 9.7 – Результат выполнения тестов

Результаты тестов показывают, что метод обработки запроса работает корректно как при корректных, так и при некорректных данных, а также в случае возникновения ошибок при получении списка пользователей.

ЗАКЛЮЧЕНИЕ

В рамках курсового проекта была разработана система управления складским учётом и регистрации заказов. Приложение состоит из двух частей: серверной и клиентской. Пользователи отправляют запросы с клиентской части на серверную, где происходит взаимодействие с базой данных и формирование ответов. Были выполнены следующие задачи: изучение предметной области, описание процесса управления складским учетом и регистрации заказов, создание и описание информационной модели, построение моделей представления на основе UML, выбор и обоснование использования программных инструментов для разработки, описание алгоритмов, реализующих бизнес-логику системы, составление руководства пользователя и тестирование системы.

В целях безопасности пользователи были разделены на три роли: пользователь, работник склада и администратор. Каждая роль привязывается к определенной учетной записи.

Разработанная система управления складским учетом позволит упростить и автоматизировать как процесс добавления новых товаров для владельцев складов, так и процесс заказа товаров.

Теперь работники склада могут легко отслеживать наличие товаров, их распределение по различным отделам и местам хранения, а также вести учет поступления и отгрузки товаров. Администраторы имеют возможность управлять пользователями, назначать им роли и права доступа, а также мониторить работу системы в целом.

Система также предоставляет возможность генерации отчетов о состоянии склада, остатках товаров, выполненных заказах и других операциях. Это значительно упрощает процесс управления складом и позволяет принимать более обоснованные решения на основе актуальной информации.

Кроме того, разработанная система обеспечивает безопасность данных и защиту от несанкционированного доступа. Все операции с данными осуществляются через защищённое соединение, а доступ к конфиденциальной информации контролируется на уровне ролей и прав пользователей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Павлековская И. В., Староверова О. В., Уринцов А. И. Влияние научно-технического прогресса на развитие информационного общества // Вестник экономической безопасности. № 3.
- [2] Арнольд, К., Гослинг, Дж., Холмс, Д. Язык программирования Java. – 3-е изд. – М. : Вильямс, 2001. – 624 с.
- [3] Эккель, Б. Философия Java. – 4-е изд. –СПб. : Питер, 2011. –640 с.
- [4] Блох, Д. Java. Эффективное программирование. –М. : Лори, 2002. – 224 с.
- [5] Макконнелл, С. Совершенный код. –СПб. : Питер, 2005. –896 с.
- [6] Хорстманн, К. С., Корнелл, Г. Библиотека профессионала. Java 2 : Том 1. Основы. –8-е изд. –М. : Вильямс, 2013. –816 с.
- [7] Хорстманн, К. С., Корнелл, Г. Библиотека профессионала. Java 2. : Том 2. Тонкости программирования. –8-е изд. –М. : Вильямс, 2012. –992 с.
- [8] Блинов, И. Н., Романчик, В. С. Java 2. Практическое руководство. – Минск : УниверсалПресс, 2005. – 400 с.
- [9] Блинов, И. Н., Романчик, В. С. Java. Промышленное программирование. – Минск : УниверсалПресс, 2007. –704 с.
- [10] Тейт, Б. Горький вкус Java – СПб. : Питер, 2003. – 334 с.
- [11] Буч, Г., Рамбо, Дж., Джекобсон, А. Язык UML. Руководство пользователя. – М. : ДМК, 2000. – 432 с.
- [12] Фаулер, М. UML. Основы. – 3-е изд. – СПб. : Символ-плюс, 2006. – 192 с.
- [13] Ларман, К. Применение UML 2.0 и шаблонов проектирования. Введение в объектно-ориентированный анализ и проектирование. – 3-е изд. – СПб. : Вильямс, 2012. – 736 с.
- [14] Стелтинг, С., Маасен, О. Java. Применение шаблонов Java. – М. : Вильямс, 2002. – 576 с.
- [15] Гамма, Э., Хелм, Р., Джонсон, Р., Влиссидес, Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб. : Питер, 2007. – 366 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Отчёт о проверке на заимствования в системе «Антиплагиат»

ОТЧЕТ №1 Проверено: 10.12.2023 18:47:10



Начало проверки: 10.12.2023 18:46:49

Длительность проверки: 00:00:20

Модуль поиска

Совпадения

Самоцитирования

Цитирования

Оригинальность

Интернет Free

11.95%

0%

0%

88.05%

Рисунок А.1 – Результат проверки на антиплагиат

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг кода алгоритмов, реализующих бизнес-логику

Функция создания заказа

```
public boolean AddNewOrder(Order order) {
    if (connection == null) {
        connection = JDBCConnector.GetConnection();
    }
    try {
        PreparedStatement preparedStatement =
connection.prepareStatement(ProductQueries.addOrder.toString());
        PreparedStatement idProducts =
connection.prepareStatement(ProductQueries.findIdProduct.toString());
        idProducts.setString(1, order.getNameProduct());
        ResultSet resultSet = idProducts.executeQuery();
        if(resultSet.next())
        {
            preparedStatement.setInt(1,
resultSet.getInt("idProducts"));
        }
        preparedStatement.setInt(2, order.getAmount());
        preparedStatement.setDate(3, order.getDate());
        preparedStatement.setDouble(4, order.getCost());
        PreparedStatement idCustomer =
connection.prepareStatement(ProductQueries.findIdCustomer.toString());
        idCustomer.setString(1, order.getNameCustomer());
        ResultSet resultSet1 = idCustomer.executeQuery();
        if(resultSet1.next())
        {
            preparedStatement.setInt(5,
resultSet1.getInt("idCustomers"));
        }
        else
        {
            PreparedStatement idSuppliers1 =
connection.prepareStatement(ProductQueries.addCustomer.toString());
            idSuppliers1.setString(1, order.getNameCustomer());
            idSuppliers1.setString(2, order.getCountry());
            idSuppliers1.setString(3, order.getAdress());
            idSuppliers1.executeUpdate();
            PreparedStatement idCustomer1 =
connection.prepareStatement(ProductQueries.findIdCustomer.toString());
            idCustomer1.setString(1, order.getNameCustomer());
            ResultSet resultSet2 = idCustomer1.executeQuery();
            if(resultSet2.next())
```

Продолжение приложения Б

```
        {
            preparedStatement.setInt(5,
resultSet2.getInt("idCustomers"));
        }
        return preparedStatement.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
```

Функция подтверждения заказа

```
public boolean AddNewOrder(CompletedOrder completedOrder) {
    if (connection == null) {
        connection = JDBCConector.GetConnection();
    }
    try {
        PreparedStatement preparedStatement =
connection.prepareStatement(CompletedOrdersQueries.insertOrder.toString());

        PreparedStatement idProducts =
connection.prepareStatement(ProductQueries.findIdProduct.toString());
        idProducts.setString(1, completedOrder.getNameProduct());
        ResultSet resultSet = idProducts.executeQuery();
        if(resultSet.next())
        {
            preparedStatement.setInt(1,
resultSet.getInt("idProducts"));
        }
        preparedStatement.setInt(2, completedOrder.getAmount());
        preparedStatement.setDate(3,
completedOrder.getOrderDate());
        preparedStatement.setDate(4,
completedOrder.getConfirmationDate());
        preparedStatement.setDouble(5, completedOrder.getCost());
        PreparedStatement idCustomer =
connection.prepareStatement(ProductQueries.findIdCustomer.toString());
        idCustomer.setString(1, completedOrder.getNameCustomer());
        ResultSet resultSet1 = idCustomer.executeQuery();
        if(resultSet1.next())
        {
            preparedStatement.setInt(6,
resultSet1.getInt("idCustomers"));
        }
        Else
```

Продолжение приложения Б

```
{
    PreparedStatement idSuppliers1 =
connection.prepareStatement(ProductQueries.addCustomer.toString());
    idSuppliers1.setString(1,
completedOrder.getNameCustomer());
    idSuppliers1.setString(2,
completedOrder.getCountry());
    idSuppliers1.setString(3, completedOrder.getAdress());
    idSuppliers1.executeUpdate();
    PreparedStatement idCustomer1 =
connection.prepareStatement(ProductQueries.findIdCustomer.toString());
    idCustomer1.setString(1,
completedOrder.getNameCustomer());
    ResultSet resultSet2 = idCustomer1.executeQuery();
    if(resultSet2.next())
    {
        preparedStatement.setInt(6,
resultSet2.getInt("idCustomers"));
    }

}
return preparedStatement.executeUpdate() > 0;
} catch (SQLException e) {
    e.printStackTrace();
    return false;
}
}
```

Функция добавления товара

```
public boolean InsertNewProduct(Product product) {
    if (connection == null) {
        connection = JDBCConector.GetConnection();
    }
    try {
        PreparedStatement preparedStatement =
connection.prepareStatement(ProductQueries.insertProduct.toString());
        PreparedStatement idSuppliers =
connection.prepareStatement(ProductQueries.findIdSupplier.toString());
        idSuppliers.setString(1,product.getSupplier());
        ResultSet resultSet = idSuppliers.executeQuery();
        if(resultSet.next())
        {
            preparedStatement.setInt(1,
resultSet.getInt("idSuppliers"));
        }
        else
```

Продолжение приложения Б

```
{
    PreparedStatement idSuppliers1 =
connection.prepareStatement(ProductQueries.addSupplier.toString());
    idSuppliers1.setString(1,product.getSupplier());
    idSuppliers1.executeUpdate();
}
    PreparedStatement idGroupProduct =
connection.prepareStatement(ProductQueries.findIdProductsGroup.toStrin
g());
    idGroupProduct.setString(1,product.getProductsGroup());
    ResultSet resultSet1 = idGroupProduct.executeQuery();
    if(resultSet1.next())
    {
        preparedStatement.setInt(2,
resultSet1.getInt("idProductsGroups"));
    }
    else
    {
        PreparedStatement idGroupProduct1 =
connection.prepareStatement(ProductQueries.addProductGroup.toString())
;

idGroupProduct1.setString(1,product.getProductsGroup());
        idGroupProduct1.executeUpdate();
    }
    preparedStatement.setString(3, product.getName());
    preparedStatement.setDouble(4,product.getPrice());
    preparedStatement.setInt(5, product.getAmount());
    java.util.Date utilDate = product.getReceiptDate();
    java.sql.Date sqlDate = new
java.sql.Date(utilDate.getTime());
    preparedStatement.setDate(6, sqlDate);
    return preparedStatement.executeUpdate() > 0;
} catch (SQLException e) {
    e.printStackTrace();
    return false;
}
}
```

Функция, возвращающая список завершённых заказов

```
public ArrayList<CompletedOrder> GetCompletedOrders(){
    if (connection == null) {
        connection = JDBCConnector.GetConnection();
    }
    ArrayList<CompletedOrder> completedOrders = new ArrayList<>();
    try {
```

Продолжение приложения Б

```
        PreparedStatement preparedStatement =
connection.prepareStatement(CompletedOrdersQueries.getOrders.toString(
));
        ResultSet resultSet = preparedStatement.executeQuery();
        while(resultSet.next()){
            completedOrders.add(SetOrderFromResultSet(resultSet));
        }
    } catch (SQLException e) {
        e.printStackTrace();
        return null;
    }
    System.out.println(completedOrders);
    return completedOrders;
}
```

Функция, которая достаёт данные о заказе из ResultSet

```
private CompletedOrder SetOrderFromResultSet(ResultSet resultSet){
    CompletedOrder order = new CompletedOrder();
    try {
        order.setAmount(resultSet.getInt("amount"));
        order.setCost(resultSet.getDouble("cost"));

order.setNameProduct(resultSet.getString("products.name"));

order.setNameCustomer(resultSet.getString("customers.name"));
        order.setOrderDate(resultSet.getString("orderDate"));

order.setConfirmationDate(resultSet.getString("confirmationDate"));
    } catch (SQLException e) {
        e.printStackTrace();
    } catch (ParseException e) {
        throw new RuntimeException(e);
    }
    return order;
}
```